Week 10: Objects, transactions and security

Learning outcomes: CLO1, CLO4

Lesson

- **View:** saved query used like a table. **Index:** faster lookups, costs space and write time; check with `EXPLAIN`.
- **Procedure:** named SQL you `CALL`. **Trigger:** runs on INSERT/UPDATE/DELETE. Create them in `student_work`.
- **Transaction:** commit or roll back changes together. A failed statement is usually cancelled alone and the transaction stays open; a deadlock rolls back everything. DDL commits automatically.
- **ACID: atomicity** (all changes or none), **consistency** (rules still hold), **isolation** (transactions don't interfere), **durability** (committed changes survive a crash).
- **Least privilege:** give each account only what it needs. `practice` can read `cambodia` and change `student_work`.
- **Three levels:** external (views), conceptual (tables), internal (storage). **Physical independence:** change storage without changing tables. **Logical independence:** change tables while views stay the same.

Worked example

Run `example.sql` (how (`setup/README.md`)), then answer in `work.sql`.

Tables for this week

Read `cambodia`, write `student_work`. All columns are `NOT NULL`. Samples are the first two rows by code.

`cambodia.province`

Column	Type	Key	Meaning
`province_pcode`	`CHAR(4)`	PK	Province code
`province_name`	`VARCHAR(100)`		Province name (English)
`area_km2`	`DECIMAL(12,1)`		Area in km², one decimal place

Sample rows:

province_pcode	province_name	area_km2
KH01	Banteay Meanchey	6145.9
KH02	Battambang	11869.0

`cambodia.district`

Column	Type	Key	Meaning
`district_pcode`	`CHAR(6)`	PK	District code
`district_name`	`VARCHAR(100)`		District name (English)
`area_km2`	`DECIMAL(12,1)`		Area in km², one decimal place
`province_pcode`	`CHAR(4)`	FK	Province code

Sample rows:

district_pcode	district_name	area_km2	province_pcode
KH0102	Mongkol Borei	807.2	KH01
KH0103	Phnum Srok	761.7	KH01

Join keys: `district.province_pcode = province.province_pcode`.

Created by `example.sql` in `student_work`:

Object	Columns or parameters
`w10_province` (table)	Copy of `cambodia.province`
`w10_audit` (table)	`province_pcode CHAR(4)`, `deleted_at TIMESTAMP(6)`
`w10_district_counts` (view)	`province_pcode`, `district_count`
`w10_districts` (procedure)	Input `code CHAR(4)`; returns `district_pcode`, `district_name`

More: data guide (`data/README.md`).

Lab tasks

1. Create a view of district counts per province in `student_work`.
2. Run `CREATE TABLE student_work.w10_district AS SELECT * FROM cambodia.district;` (no indexes copied). Check `SHOW INDEX`, run EXPLAIN on a `province_pcode` filter, add an index, run EXPLAIN again.
3. Open two sessions and run `START TRANSACTION;` in both. In each, update `student_work.w10_province` where `province_pcode = 'KH01'`, setting the name to its current value. Do A first and don't commit: B waits. Roll back A, then B.
4. Create a procedure that lists districts for an input province code; call it.
5. Create `lab10_province` (copy of provinces), `lab10_audit` and an AFTER DELETE trigger `lab10_deleted` that logs deletes. Delete a row in a transaction, check the audit table, roll back.
6. Self-study: run `SHOW GRANTS;`. On paper, write CREATE USER, GRANT and REVOKE for a SELECT-only app account (`practice` can't run them).

Homework

1. Explain tasks 3 and 5 with ACID.
2. Submit your view, index, procedure and trigger scripts with output, plus your written GRANT/REVOKE. Why is the audit row rolled back too?
3. Use your view to explain the three levels and data independence.

Capstone sprint

C10: Add the views, indexes, procedures or triggers your project needs; describe your app account's permissions.

Submit

Answers in `work.sql`, reflection in `reflection.md`, output in `results/`. Commit and push. See SUBMISSIONS.md.